

Based on what I know about you and your current skills:

- You're comfortable with **Python and Flask**.
- You're currently learning **n8n**.
- You have experience with **Google Sheets, Render, SQLite/PostgreSQL**, and **email integrations**.
- This is a **personal internal tool**, so we don't need production-level architecture.

That means Relay can be built much faster than FlowSphere.

Estimated Timeline

Phase	Duration
Planning & Database	1 day
Attendance Module	2 days
Worksheet Tracker	2 days
Google Sheets Integration	2 days
n8n Workflows	3-4 days
Dashboard (Basic UI)	3 days
Reports & Logging	2 days
Testing & Bug Fixes	3-4 days

Total: 18-20 working days

Assuming you spend **3-4 hours per day**.

If you're learning while building

Since you're also learning n8n, expect:

25-30 days

because you'll spend time understanding concepts, experimenting, and fixing mistakes.

I recommend building it in 4 milestones

Milestone 1 (Week 1)

Goal: Save 30% of your daily work.

Features:

- Login (if needed)
- Google Sheets connection
- Attendance checker
- Worksheet checker
- Basic database

At the end of Week 1, Relay is already useful.

Milestone 2 (Week 2)

Goal: Save 60–70% of your work.

Features:

- n8n workflows
- Email automation
- Reminder generation
- Logging
- Daily reports

Now Relay starts working for you instead of just displaying information.

Milestone 3 (Week 3)

Goal: Make it reliable.

Features:

- Better UI
 - Error handling
 - Scheduled workflows
 - Dashboard improvements
 - Bug fixes
-

Milestone 4 (Week 4)

Goal: Polish.

Features:

- Statistics
 - Export reports
 - Performance improvements
 - Code cleanup
 - Documentation
-

Don't try to automate everything first

One mistake many people make is trying to automate every task before they have a working tool.

Instead, use the **80/20 rule**:

Identify the 20% of tasks that consume 80% of your time.

For example:

- Attendance → 10 minutes/day
- Worksheet checking → 20 minutes/day
- Sending reminders → 15 minutes/day
- Preparing reports → 15 minutes/day

If Relay automates just those, you've already reclaimed nearly an hour each day. Everything else can be added later if it's genuinely worth it.

My proposal

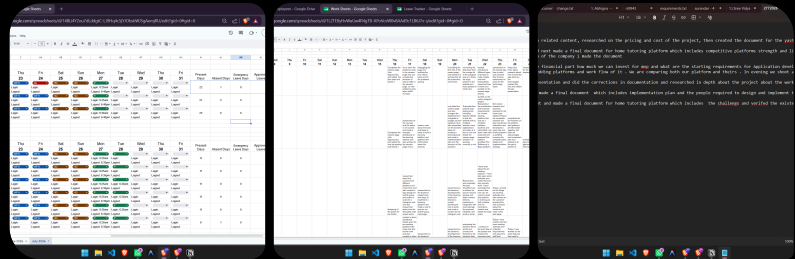
Since we've already planned your **FlowSphere** journey, I'd treat **Relay** as a **30-day build challenge**.

We'll build it incrementally:

- **Week 1:** Foundation
- **Week 2:** Automation
- **Week 3:** Intelligence and reporting
- **Week 4:** Polish and real-world testing

By the end of those 30 days, you'll have:

- A tool that meaningfully reduces your workload.
- Hands-on experience with n8n and workflow automation.
- A much clearer understanding of what should—and shouldn't—go into FlowSphere.



i will tell my motive again.

juniors have to enter login time and logout time.

based on that attendance should happen.

worksheet will be sperate google sheet , they have to fill it within specific time, if don't their worksheet status will be not updated(like that). ALSO i will attach my current sheet which will be helpful for you.

The last pic is like their worksheet i need to send to my ceo. This is what i'm doin at present. this work only i need to automate

Stopped thinking

Now I understand exactly what **Relay** should be.

And I have to say—**this is much smaller than I initially imagined.** This isn't a workflow platform. It's a **personal automation assistant** for your daily reporting.

After looking at your sheets, here's what I understand.

Current Process (Manual)

1. Attendance Sheet

Every junior enters:

- Login Time
- Logout Time

- Status (P / WFH / HD / Ab etc.)

Then the sheet calculates:

- Present Days
- Absent Days
- Emergency Leave
- Approved Leave
- Weekend Days
- Attendance %

This sheet already does most of the calculations.

2. Worksheet Sheet

Every junior writes today's work.

Example:

Completed careers page...

Fixed backend bugs...

Researched project...

Each date has one cell.

3. Your Work

This is the repetitive part.

You open every student's worksheet.

Read today's cell.

Then manually type something like:

27/07/2026

1. Phanichand -
explained project expo content...
2. Bhavana -
prepared PPT...

3. Yashwanth -
researched MVP...

Then you send it to your CEO.

This is the work that consumes your time.

Relay should automate ONLY this.

Nothing more.

Relay v1 Features

Module 1

Attendance Checker

Read attendance sheet.

Show:

Present : 18

Absent : 2

Late : 1

WFH : 4

Done.

Module 2

Worksheet Checker

At 6:30 PM

Check every employee.

If today's worksheet cell is empty

↓

Worksheet Status

✗ Not Updated

Otherwise

✓ Updated

No manual checking.

Module 3 (Most Important)

Generate CEO Report

This is where Relay saves you 80% of your time.

Input:

Worksheet Sheet

↓

AI reads today's entries.

↓

Output

27/07/2026

1. Phanichand

- Explained project expo content.
- Researched pricing.
- Prepared documentation.

2. Bhavana

- Completed PPT.
- Finalized tutoring platform document.

3. Yashwanth

- Studied MVP planning.
- Compared competitors.

...

Exactly like you currently prepare manually—but generated automatically from the worksheet entries.

Module 4

Export

One button.

Generate Report



Copy

or

Download TXT

or

Download PDF

That's all.

Notice there is:

- ✗ No login system
- ✗ No employee dashboard
- ✗ No admin panel
- ✗ No complex UI

Because YOU are the only user.

Technology

Honestly...

You don't even need Flask for Version 1.

You could build it with:

n8n

↓

Google Sheets

↓

Gemini API

↓

Python Script

↓

TXT/PDF

That's enough.

Workflow

Google Sheets

Attendance

|
|

Worksheet

|
▼

Relay

|

Reads today's column

|

Checks missing worksheet

|

Calculates attendance



Gemini summarizes work



Creates report



TXT / PDF



Send to CEO

Time Estimate

Now that I know the exact scope:

Task	Time
Google Sheets Integration	1 day
Attendance Reading	1 day
Worksheet Status Checker	1 day
AI Report Generator	2 days
Export TXT/PDF	1 day
Testing	2 days

Total

Around 7-10 days, assuming you're spending a few hours each day and learning as you go.

One improvement I'd make

Looking at your process, I think there's an opportunity to eliminate even more manual work.

Instead of opening a text editor and copying the report, Relay could produce something like this automatically every evening:

 Daily Report - 28/07/2026

 Attendance

Present : 22

Absent : 1

WFH : 4

 Worksheet Status

Updated : 21

Pending : 2

Pending Employees

- Rahul
- Sai

 Work Summary

1. Phanichand

- Researched project pricing.
- Prepared Expo documentation.

2. Bhavana

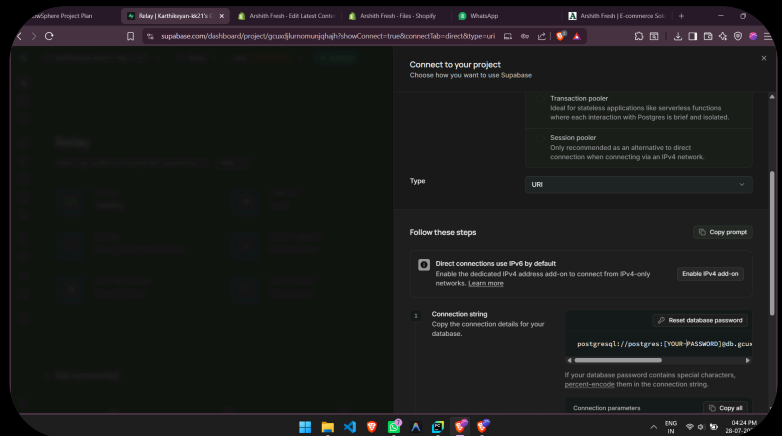
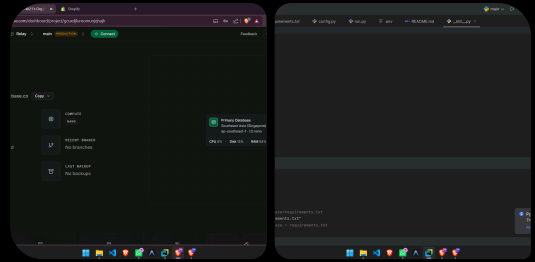
- Finalized tutoring platform PPT.

3. Yashwanth

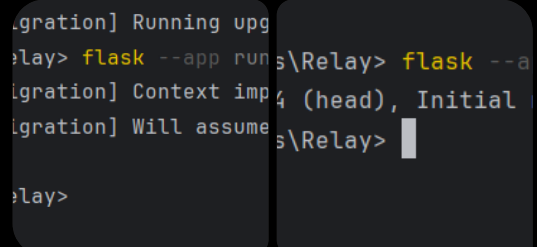
- Studied MVP architecture.
- Compared competing platforms.

...

Then, with a single click (or even automatically at a scheduled time), Relay could email or copy this report for you. That would reduce your daily effort to reviewing the generated report instead of compiling it manually.



i replaced my password and pasted it in .env file



no changes in schema detected

```
1. from flask import Flask
from config import Config
from app.extensions import db, migrate
```

```
def create_app():
    app = Flask(__name__)
    app.config.from_object(Config)
```

```
db.init_app(app)
migrate.init_app(app, db)
```

```
from app.models import User
```

```
@app.route("/")
def home():
    return "🚀 Relay Backend is Running
Successfully!"
```

```
return app
```

2. from app.extensions import db

```
class User(db.Model):
    __tablename__ = "users"

    id = db.Column(db.Integer, primary_key=True)
    employee_id = db.Column(db.String(20),
unique=True, nullable=False)
    full_name = db.Column(db.String(100),
nullable=False)
    email = db.Column(db.String(120), unique=True,
nullable=False)
    password_hash = db.Column(db.Text,
nullable=False)
    role = db.Column(db.String(20), default="user")
    active = db.Column(db.Boolean, default=True)

    created_at = db.Column(db.DateTime,
server_default=db.func.now())
```

3. from .user import User

4. from app import create_app

```
app = create_app()
```

```
if __name__ == "__main__":
    app.run(debug=True)
```

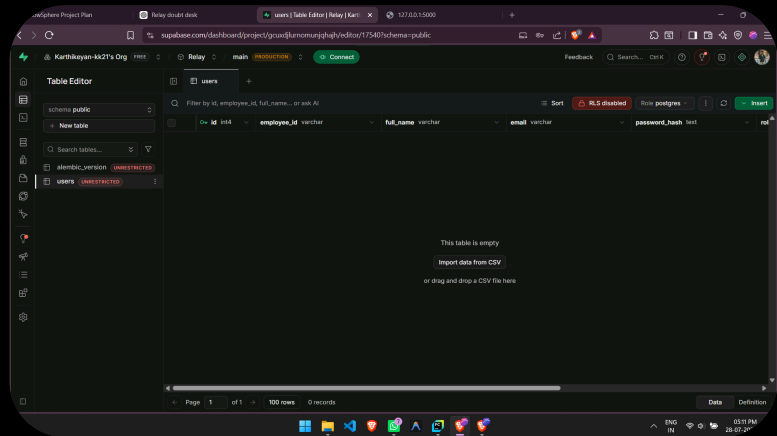
```
Please edit configuration/connection/logging settings in D:\Projects\Relay\migration
(.venv) PS D:\Projects\Relay> flask --app run.py db migrate -m "Initial migration"
INFO [alembic.runtime.migration] Context impl PostgresqlImpl.
INFO [alembic.runtime.migration] Will assume transactional DDL.
ERROR [flask_migrate] Error: Can't locate revision identified by 'e26300a84454'
(.venv) PS D:\Projects\Relay>
```

```

[alembic.env] No changes in schema detected.
(.venv) PS D:\Projects\Relay> flask --app run.py shell
Python 3.13.1 (tags/v3.13.1:0671451, Dec 3 2024, 19:06:28) [MSC v.1942 64 bit (AMD64)] on win32
App: app
Instance: D:\Projects\Relay\instance
>>> from app.extensions import db
>>>
>>> print(db.metadata.tables.keys())
dict_keys(['users'])

```

then the issue is with alembic



1. import logging

```
from logging.config import fileConfig
```

```
from flask import current_app
```

```
from alembic import context
```

```
# this is the Alembic Config object, which provides
# access to the values within the .ini file in use.
```

```
config = context.config
```

```
# Interpret the config file for Python logging.
```

```
# This line sets up loggers basically.
```

```
fileConfig(config.config_file_name)
```

```
logger = logging.getLogger('alembic.env')
```

```
def get_engine():
```

```
    try:
```

```
        # this works with Flask-SQLAlchemy<3 and
        Alchemical
```

```
        return
```

```
    current_app.extensions['migrate'].db.get_engine()
```

```
    except (TypeError, AttributeError):
```

```

        # this works with Flask-SQLAlchemy>=3
        return
current_app.extensions['migrate'].db.engine

def get_engine_url():
    try:
        return
get_engine().url.render_as_string(hide_password=False).replace(
    '%', '%%')
    except AttributeError:
        return str(get_engine().url).replace('%', '%%')

```

```

# add your model's MetaData object here
# for 'autogenerate' support
# from myapp import mymodel
# target_metadata = mymodel.Base.metadata
config.set_main_option('sqlalchemy.url',
get_engine_url())
target_db = current_app.extensions['migrate'].db

```

```

# other values from the config, defined by the
needs of env.py,
# can be acquired:
# my_important_option =
config.get_main_option("my_important_option")
# ... etc.

```

```

def get_metadata():
    if hasattr(target_db, 'metadatas'):
        return target_db.metadatas[None]
    return target_db.metadata

```

```

def run_migrations_offline():
    """Run migrations in 'offline' mode.

```

This configures the context with just a URL
 and not an Engine, though an Engine is
 acceptable

here as well. By skipping the Engine creation we don't even need a DBAPI to be available.

Calls to `context.execute()` here emit the given string to the script output.

```
"""
url = config.get_main_option("sqlalchemy.url")
context.configure(
    url=url, target_metadata=get_metadata(),
    literal_binds=True
)

with context.begin_transaction():
    context.run_migrations()
```

```
def run_migrations_online():
    """Run migrations in 'online' mode.
```

In this scenario we need to create an Engine and associate a connection with the context.

```
"""

# this callback is used to prevent an auto-
migration from being generated
# when there are no changes to the schema
# reference:
http://alembic.zzzcomputing.com/en/latest/cookbook.html
def process_revision_directives(context, revision,
directives):
    if getattr(config.cmd_opts, 'autogenerate',
False):
        script = directives[0]
        if script.upgrade_ops.is_empty():
            directives[:] = []
            logger.info('No changes in schema
detected.')

conf_args =
```

```

current_app.extensions['migrate'].configure_args
    if conf_args.get("process_revision_directives") is
None:
        conf_args["process_revision_directives"] =
process_revision_directives

connectable = get_engine()

with connectable.connect() as connection:
    context.configure(
        connection=connection,
        target_metadata=get_metadata(),
        **conf_args
    )

    with context.begin_transaction():
        context.run_migrations()

if context.is_offline_mode():
    run_migrations_offline()
else:
    run_migrations_online()

2. no .py file inside versions

```

Perfect. I think this is the best investment we can make for the project.

Instead of a conversation transcript, we'll create **Relay v1 Documentation**—the same kind of documentation a software company maintains before and during development.

Relay v1 - Software Design & Development Document

Estimated length: **80-120 pages**

This won't be a generic document. It will be written specifically for **your Relay project**.

Proposed Table of Contents

Part 1 - Project Overview

- Project Introduction

- Business Problem
 - Why Relay was built
 - Objectives
 - Current manual workflow
 - Proposed automated workflow
-

Part 2 - Requirements

- Functional Requirements
 - Non-functional Requirements
 - User Roles
 - System Modules
 - Scope
 - Out of Scope (v2 features)
-

Part 3 - System Architecture

- Overall Architecture
 - Folder Structure
 - Flask Architecture
 - Module Responsibilities
 - Request Flow
 - Authentication Flow
 - Attendance Flow
 - Worksheet Flow
-

Part 4 - Database Design

- ER Diagram
 - Users Table
 - Attendance Table
 - Worksheets Table
 - Logs
 - Settings
 - Relationships
 - Indexes
-

Part 5 - UI/UX Design

- Login Page
 - Dashboard
 - Attendance Screen
 - Worksheet Screen
 - Admin Panel
 - Reports
 - Mobile Responsiveness
-

Part 6 - Feature Specifications

Each feature documented separately.

For example:

Authentication

- Objective
- Flow
- Validations
- Database interactions
- Error handling

Attendance

- Check In
- Check Out
- Office IP validation
- Duplicate prevention

Worksheet

- Submission
 - Editing
 - Lock after 6:30 PM
 - Admin unlock
-

Part 7 - AI Module

- Gemini Integration
- Prompt Design
- CEO Report Format
- Report Generation Flow

Part 8 - n8n Automation

- Workflow diagrams
 - Triggers
 - Nodes
 - Integrations
 - Scheduling
 - Error handling
-

Part 9 - Deployment

- Environment Variables
 - Render Setup
 - Supabase Setup
 - Production Checklist
-

Part 10 - Security

- Password hashing
 - Sessions
 - CSRF
 - SQL Injection prevention
 - Authentication
 - Authorization
-

Part 11 - Testing

- Unit Tests
 - Manual Test Cases
 - User Acceptance Testing
 - Deployment Testing
-

Part 12 - Future Roadmap

Relay v2 ideas

- Leave Management
- Notifications
- Mobile App

- Analytics
 - Calendar
 - Email reminders
-

Part 13 - Daily Development Log

Every milestone will be recorded like:

```
Day 1
-----
Project Created

Completed
✓ Flask Setup
✓ Supabase
✓ SQLAlchemy

Issues
-----
Alembic Migration

Solution
-----
...

Lessons
-----
...
```

By the time we finish Relay, this becomes a complete development history.

Documentation Standards

The document will include:

- Professional formatting
- Table of Contents
- Version history
- Flowcharts
- Architecture diagrams
- Database diagrams
- Folder tree
- Code snippets where useful
- Screenshots (as we build)

- Checklists
- Notes
- Best practices

Final Deliverables

When Relay is complete, you'll have:

-  **Relay Software Design & Development Guide (80-120 pages)**
-  **Software Requirements Specification (SRS)**
-  **Database Design Document**
-  **Deployment Guide**
-  **Testing Guide**
-  **Developer Handover Document**
-  **Development Log**
-  **User Manual** (for employees and admins)

This won't just help you complete Relay—it will also give you a professional project portfolio artifact that demonstrates how the system was designed and built.

One suggestion: let's build this documentation **incrementally**. After each completed milestone (Authentication, Attendance, Worksheet, etc.), I'll update the document. That way it always stays accurate and reflects the actual implementation, rather than trying to reconstruct everything at the end.

proceed

Excellent. We'll treat this exactly like a professional software project.

Relay v1 Documentation

Version: 1.0

Status: In Development

This document will become the single source of truth for Relay. Every feature we build will be reflected here.

Chapter 1 - Project Overview

1.1 Introduction

Relay is an internal web application developed to automate employee attendance, daily worksheet management, and AI-powered reporting within the organization.

The primary objective of Relay is to eliminate repetitive manual tasks currently performed by the administrator, improve data accuracy, and reduce the time spent preparing daily operational reports.

Unlike commercial SaaS products, Relay is designed exclusively for internal organizational use. Therefore, development prioritizes simplicity, reliability, and rapid delivery over unnecessary complexity.

1.2 Business Problem

The organization currently manages several operational activities manually.

Every working day the administrator:

- Tracks employee check-in times.
- Tracks employee check-out times.
- Updates attendance in Google Sheets.
- Verifies worksheet submissions.
- Identifies employees who failed to update worksheets.
- Reviews every worksheet individually.
- Prepares a summary report.
- Sends the report to the CEO.

Although each task is simple, together they consume significant time and are prone to human error.

Relay replaces these manual activities with an automated workflow.

1.3 Objectives

The primary objectives are:

- Automate attendance tracking.
- Automate worksheet submission.
- Automatically lock worksheets after 6:30 PM.
- Generate AI summaries of daily work.
- Provide real-time dashboards.

- Eliminate repetitive administrative work.
 - Maintain accurate records.
 - Prepare CEO reports with one click.
-

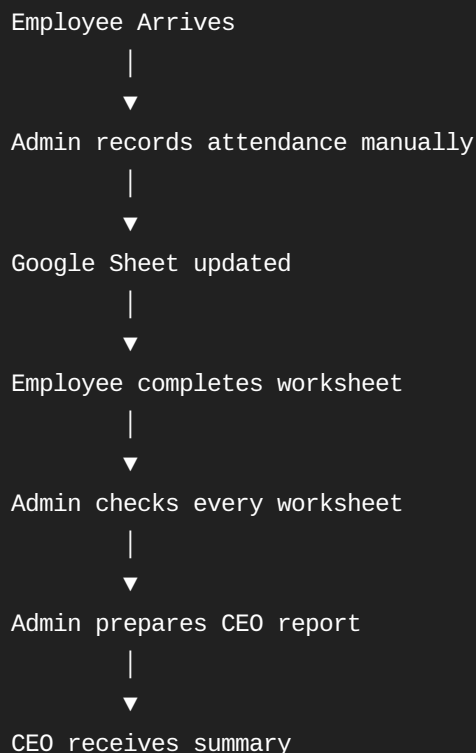
1.4 Project Vision

Relay aims to become the organization's centralized operational platform for employee activity management.

The system should:

- Require minimal manual intervention.
 - Be simple enough for employees to use without training.
 - Provide administrators with complete visibility.
 - Generate executive reports automatically.
-

1.5 Current Workflow

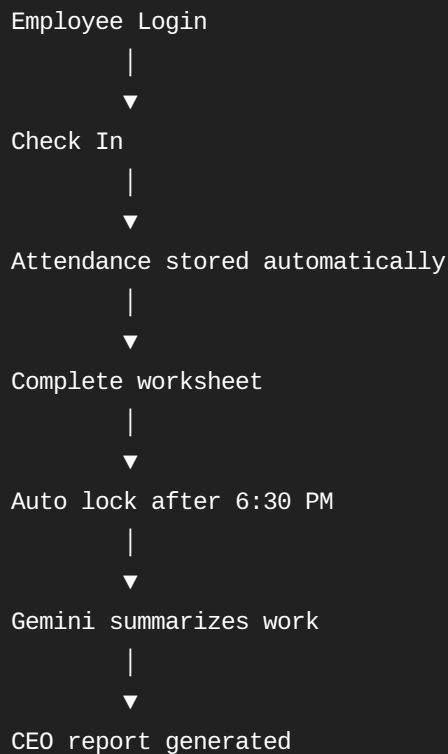


Problems:

- Time consuming
- Human errors
- Repetitive work
- Difficult to scale

- No centralized system

1.6 Proposed Workflow



Benefits:

- No manual attendance entry.
- No manual worksheet verification.
- Automatic reporting.
- Better accuracy.
- Significant time savings.

Chapter 2 - Project Scope

Included (Relay v1)

Authentication

- Login
- Logout
- Password hashing
- Session management
- Role-based access (Admin/User)

Attendance

- Check In
- Check Out
- Attendance history
- Office IP validation
- Admin corrections
- Attendance audit logs

Worksheet

- Daily worksheet
- Multiple edits before 6:30 PM
- Automatic lock at 6:30 PM
- Admin unlock
- Submission status

Dashboard

- Employee dashboard
- Admin dashboard
- Attendance overview
- Worksheet overview
- Pending tasks

AI

- Gemini integration
- CEO report generation
- Daily summary

Export

- PDF
- TXT

Excluded (Future Versions)

To ensure rapid delivery, the following features are intentionally excluded from Relay v1:

- Leave management
- Holiday calendar
- Email notifications
- Push notifications
- Mobile application

- Payroll integration
- Biometric integration
- Multi-company support
- Analytics dashboards
- Performance scoring
- Face recognition
- GPS tracking

These items will be evaluated after the successful deployment of Relay v1.

Chapter 3 - Technology Stack

Layer	Technology
Frontend	HTML, CSS, JavaScript
Backend	Flask
ORM	SQLAlchemy
Database	Supabase (PostgreSQL)
Authentication	Flask-Login
Migrations	Flask-Migrate
AI	Gemini API
Automation	n8n
Deployment	Render
Version Control	Git & GitHub

Chapter 4 - Development Philosophy

Relay follows a delivery-first approach.

Core Principles

- Keep the architecture clean.
- Avoid unnecessary complexity.

- Prefer proven solutions.
 - Freeze scope once approved.
 - Complete one module before starting another.
 - Prioritize working software over theoretical perfection.
-

Definition of Done

Relay v1 will be considered complete when:

- Admin can log in.
 - Admin can create users.
 - Employees can log in.
 - Employees can check in.
 - Employees can check out.
 - Employees can submit worksheets.
 - Worksheets lock automatically after 6:30 PM.
 - Dashboard displays real-time information.
 - AI generates CEO reports.
 - Application is deployed and used in production.
-

This is **Chapter 1-4** of what will become the complete Relay documentation. Rather than generating all 100+ pages now—before the implementation exists—we'll expand each chapter as we build the corresponding module. That way, the documentation stays synchronized with the actual system and can serve as a true reference for development, deployment, and future maintenance.

i want everything in pdf

Done!

You can download the PDF here:

 **Relay_v1_Project_Documentation_v1.pdf**

One important point

This is the **initial documentation draft** based on everything we've finalized so far. However, it is **not yet the 80-120 page complete document**!

proposed.

The reason is simple: many sections (database schema, screenshots, code snippets, API documentation, UI designs, flowcharts, deployment steps, testing evidence, etc.) don't exist yet because we haven't built those modules.

What I recommend

Instead of generating a placeholder 100-page PDF filled with repetitive content, let's create a **professional document that grows with the project**.

By the end of Relay, you'll have a document comparable to what software companies produce, including:

- 📖 100–150 pages
- 🏗️ Complete system architecture
- 📊 Database ER diagrams
- 🔄 Flowcharts for every module
- 🧩 API documentation
- 📸 UI screenshots
- 💻 Important code snippets
- 🔒 Security design
- 🚀 Deployment guide
- 📝 Test cases
- 📖 User manual
- 🚚 Developer handover guide
- 📅 Complete development log from Day 1 to deployment

That final document will be something you can:

- hand to another developer,
- use for maintenance,
- include in your portfolio,
- and use as internal technical documentation.

I recommend we keep updating it after every milestone so the final version is accurate, comprehensive, and genuinely useful rather than padded with speculative content. ☐